SidusPro™ FX软件模块使用说明

Version	V1.2
Author	Matthew
Date	2020.12.17

- **History：**

-v1.0: 2020/06/18,Matthew,初始版本
-v1.1: 2020/08/06,Matthew,修复MFX光效部分Bug
-v1.2: 2020/12/17,Matthew,新增pwm数据队列操作接口,修复部分MFX光效bug

一、接口实现

接口实现文件在SidusPro_Interface.c文件中，需要实现如下几个函数

- 随机数获取函数： 获取在min和max之间的

```
uint32_t SidusPro_Get_RandomRange(uint32_t min, uint32_t max)
{
    //ToDo
}
```

- pwm设置函数： 设置设备pwm

```
void SidusPro_Set_Pwm(SidusPro_Pwm_Type* pwm_struct)
{
    //Todo
}
```

- CCT转pwm函数： 实现CCT转pwm函数

```
void SidusPro_CCT2PWM(uint16_t cct, uint8_t gm, uint16_t lightness,
SidusPro_Pwm_Type* rgbww_pwm_struct)
{
    //Todo
}
```

- CFX光效文件搬移缓存函数： 将外部SPI Flash中对应Type和Bank光效文件缓存到片上ROM/RAM中自定义地址，返回值为缓存的片上ROM/RAM地址。在缓存完毕后并检验缓存文件无误后，回复App Ack信息，对应Command为 BT_CommandType_CFX_Ctrl，Ctrl类型为Init。

```
uint32_t SidusPro_CFX_File_Cache(uint8_t Type, uint8_t Bank)
{
    //uint32_t CacheAddr = CFX_CACHE_ADDR;
    //Todo
}
```

- pwm队列创建函数： 创建存储光效运行存放pwm数据的队列。

```
bool SidusPro_PwmQueue_Create(uint32_t count, uint32_t size)
{
    //Todo
}
```

- pwm队列put函数： 向创建的pwm队列添加一个数据。

```
bool SidusPro_PwmQueue_Put(const void * item)
{
    //Todo
}
```

- pwm队列get函数： 创建的pwm队列获取一个数据。

```
bool SidusPro_PwmQueue_Get(void * item)
{
    //Todo
}
```

- pwm队列重置函数： 重置pwm队列为创建时的状态，即为清空队列数据。

```
bool SidusPro_PwmQueue_Reset(void)
{
    //Todo
}
```

- 查询队列容量是否已满函数： 查询创建的pwm队列是否已满。

```
bool SidusPro_PwmQueue_Is_Full(void)
{
    //Todo
}
```

二、用户调用函数

需调用函数在SidusProFX.h中

- SidusProFX系统初始化函数，上电调用一次

```
uint32_t SidusProFX_System_Init(void);
```

- SidusProFX光效任务函数，需要循环调用

```
void SidusProFX_Task(void);
```

- SidusProFX一毫秒中断函数，中断中一毫秒调用一次

```
void SidusProFX_1Ms_Callback(void);
```

- SidusProFX光效参数初始化函数，运行光效时传参调用，参数类型为SidusProFX_ArgBody_Type*

```
void SidusProFX_Arg_Init(SidusProFX_ArgBody_Type* Arg);
```

- SidusProFX光效开光函数，初始化参数后传参true光效开始运行。关闭光效传false直接关闭

```
void SidusProFX_Enable(bool en);
```

- SidusProFX光效关状态查询

```
bool SidusProFX_State(void);
```

三、SidusPro CFX文件说明

一体式灯可以直接使用SidusProFile.c(需实现一些接口函数)，分离式灯体可参考以下信息接收和存储光效文件。 文件接收走GATT通道，具体可参照GATT通讯协议内容。

1、app传输bin文件格式说明

文件由128Bytes头信息 + n * 4Bytes + 16Bytes尾信息组成：

- 头信息：数据结构如下

```
typedef __packed struct
{
   uint8_t Tag[16];      //文件标识符                          ---16Byte
   uint8_t TxMode;       //传输模式 0:SidusProFX                ---1Byte
   uint16_t FxType;      //子模式 0:Picker 1:TouchBar 2:Music   ---2Byte
   uint16_t Supported;   //支持光效类型                         ---2Byte
   uint16_t UintTime;    //单位时间/采样率                      ---2Byte
   uint8_t Bars;         //小节格式                            ---1Byte
   uint32_t FrameNum;    //帧数                                ---4Byte
   uint8_t Reserve[100];//预留                               ---100Byte
} CFX_IntFileHead_Type;  //接收bin文件数据头信息              ---128Byte

//16Bytes 文件标识符: Tag[16], 固定为以下16Bytes
const uint8_t CFX_FileTag[16] = {0x8B, 0x99, 0x92, 0x3E, 0xF4, 0x90, 0x53, 0x80,
0xCA, 0xD7, 0xF9, 0x53, 0x2C, 0xF2, 0x33, 0x8D};
```

- n * 4Bytes光效数据信息：n代表帧数，4Bytes数据结构用如下union表示

```
typedef struct
{
   uint32_t Int    : 10;
   uint32_t GM     : 10;
   uint32_t CCT    : 10;
   uint32_t Mode   : 2;     //0:空 1:CCT 2:HSI
} CFX_CCT_Pack_Type;

typedef struct
{
   uint32_t Int    : 10;
   uint32_t Sat    : 10;
   uint32_t Hue    : 10;
   uint32_t Mode   : 2;     //0:空 1:CCT 2:HSI
} CFX_HSI_Pack_Type;

typedef union
{
   CFX_CCT_Pack_Type CCT_Pack;
   CFX_HSI_Pack_Type HSI_Pack;
} CFX_Pack_Type;
```

- 16Bytes尾信息：固定为以下16Bytes

```
const uint8_t CFX_FileTag[16] = {0x8B, 0x99, 0x92, 0x3E, 0xF4, 0x90, 0x53, 0x80,
0xCA, 0xD7, 0xF9, 0x53, 0x2C, 0xF2, 0x33, 0x8D};
```

2、本地存储光效文件格式说明

文件由128Bytes头信息 + n * SidusPro_Pwm_Type + 16Bytes尾信息组成：

- 头信息：数据结构如下

```
typedef __packed struct
{
    uint8_t Tag[16];     //文件标识符                          ---16Byte
    uint8_t TxMode;      //传输模式 0:SidusProFX                ---1Byte
    uint16_t FxType;     //子模式 0:Picker 1:TouchBar 2:Music   ---2Byte
    uint16_t Supported; //支持光效类型                         ---2Byte
    uint16_t UintTime;  //单位时间/采样率                       ---2Byte
    uint8_t Bars;        //小节格式                             ---1Byte
    uint32_t FrameNum;  //帧数                                 ---4Byte
    uint8_t Reserve[95];//头信息大小                           ---95Byte
    uint8_t FxBank;      //Bank区域                             ---1Byte
    uint32_t PwmDataCrc;//pwm数据crc校验                       ---4Byte
} CFX_PwmFileHead_Type; //本地存储pwm数据头信息              ---128Byte

//16Bytes 文件标识符：Tag[16]，固定为以下16Bytes
const uint8_t CFX_FileTag[16] = {0x8B, 0x99, 0x92, 0x3E, 0xF4, 0x90, 0x53, 0x80,
0xCA, 0xD7,  0xF9, 0x53, 0x2C, 0xF2, 0x33, 0x8D};
```

- n * SidusPro_Pwm_Type光效数据信息：n代表帧数

```
typedef __packed struct
{
    uint16_t R;
    uint16_t G;
    uint16_t B;
    uint16_t WW;
    uint16_t CW;
} SidusPro_Pwm_Type;
```

- 16Bytes尾信息：固定为以下16Bytes

```
const uint8_t CFX_FileTag[16] = {0x8B, 0x99, 0x92, 0x3E, 0xF4, 0x90, 0x53, 0x80,
0xCA, 0xD7, 0xF9, 0x53, 0x2C, 0xF2, 0x33, 0x8D};
```

3、光效文件接收转换与存储

app接收bin文件与本地存储光效信息结构相似。128Bytes头信息中，本地存储光效多了PwmDataCrc用于校验文件完整性和区分唯一性。中间光效数据信息，本地存储文件将INT、CCT、GM等转换为具体的pwm信息，方便光效运行过程中快速执行。PwmDataCrc校验方式与OTA升级一致，检验部分为去掉128Bytes头信息和16Bytes文件末尾标识符后剩余的n * SidusPro_Pwm_Type这部分数据。文件通过GATT发送到设备端，设备端接收过程中将文件格式转换为本地存储信息，即将中间部分每一帧的INT、CCT、GM信息转换为pwm。最后计算检验值存储。可参考SidusProFile.c中实现方法。

4、光效文件管理

由于App需要读取设备端的光效Bank信息，即当前Type下Bank1-Bank10是否存在光效信息，所以设备端需要对本地光效文件进行管理。

1. 系统上电，快速校验光效文件信息并生记录本机光效文件存储信息（Type对应Bank下有无光效文件），只看头尾不计算校验值(所有光效都是2Min的情况下计算所有文件校验值时间过长)
2. GATT接收完一个完整光效后置位当Bank存储信息
3. 软件擦除对应Bank后清零当前Bank信息

5、光效文件缓存

CFX光效在运行的时候需要从外部SPI flash读取文件缓存到mcu片上Rom/Ram中，app控制时，还需要回复缓存完毕ack信息。所以需要实现接口中的最后一个函数。当缓存地址为片上ROM时，为了保护片上Flash寿命，在重复缓存同一光效文件时不应擦除当前缓存文件，应在判断外部SPI Flash中对应文件与当前缓存区文件是否一致后再做擦除动作。文件一致性判断条件为FxType相同、FxBank相同、FrameNum相同、PwmDataCrc相同，这几项与外部SPI Flash中文件相同且缓存区文件crc计算校验通过时，可不重复擦除进行缓存。

四、模块文件